Chapter 3 — Code Review

How we run reviews, why we run them this way, and what to expect as both author and reviewer.

Code review is the single most leveraged process in our engineering culture. It is the moment at which an idea, partially formed in one engineer's head, becomes a shared artefact that the rest of the team can reason about, critique, and ultimately depend on. A good review process treats this moment with the seriousness it deserves; a bad one treats it as a procedural hurdle to be cleared as quickly as possible. The difference, over the lifetime of a codebase, is enormous.

The first principle is that a code review is a conversation, not an inspection. The author has context the reviewer does not, and the reviewer has perspective the author cannot easily access from inside their own change. Both of these are valuable, and the goal of the review is to surface the parts of each that are most useful to the decision at hand. When reviewers approach a change as a problem to be solved together rather than a verdict to be issued, the quality of the resulting code rises sharply.

The second principle is that the review should be sized to fit a human attention span. A change that takes more than thirty minutes to read carefully is a change that will not be reviewed carefully. Splitting work into smaller, independently meaningful pieces is not a stylistic preference — it is the single most reliable lever an author has for getting useful feedback. We accept the small overhead of multiple reviews because the alternative is one large review that is, in practice, no review at all.

The third principle is that the reviewer's job is to ask questions, not to dictate solutions. A reviewer who writes "do it this way instead" is exercising a kind of power that often produces compliance rather than insight. A reviewer who writes "what happens when X is null here?" or "have we considered the case where two clients race on this update?" is doing something far more valuable: they are putting the author in a position to discover the right answer themselves, with full ownership of the result.

The fourth principle is that approval is not a rubber stamp. When you click approve, you are stating that you have read the change carefully enough to defend the decisions it embodies if something goes wrong six months from now. This is a meaningful claim, and it is the reason approvals carry weight. If you have not done the reading required to make that claim, do not approve — request more time, ask another reviewer to share the load, or say plainly that you have skimmed it but have not done a deep read.

Authors have responsibilities too. The most important is to make the change easy to review: a clear description of the problem, a summary of the approach, links to relevant prior discussion, and — crucially — a thoughtful self-review pass before requesting review from anyone else. Reviewers who open a change to find unrelated edits, dead code, or commented-out experiments tend to lose trust quickly, and that lost trust takes a long time to rebuild.

A frequent question is how to handle disagreement. Our default is that the reviewer raises the concern, the author responds, and if the two cannot reach agreement within a round or two, a third engineer is invited in to break the tie. This works because everyone has agreed in advance that disagreement is normal and that the process for resolving it is bounded. Long, escalating threads are a sign that the process has broken down — when you see one forming, stop typing and pick up a video call instead.

We do not require reviews to be blocking. For small, low-risk changes — typos, dependency bumps, internal-tool tweaks — a self-merge with post-hoc review is acceptable, provided the author flags the change for review within the same day. This is not a loophole; it is an acknowledgement that not every change carries the same risk profile, and treating them all identically wastes the team's most expensive resource: focused attention.

Finally, remember that the codebase is the team's shared memory. A reviewer who lets through a change that future engineers will struggle to understand is borrowing from a fund that only ever pays back with interest. A few extra minutes spent during review — pushing for clearer names, simpler control flow, better-placed comments — saves hours of confusion later. This is the deepest reason we invest in review: not because we distrust authors, but because we trust the future readers, and we want to leave them something they can work with.

— end of chapter —